

Mock questions for Final preparation

In the final exam, you can expect analogous questions to the following questions. I recall that the list serves as an example illustrating the types of questions you may encounter. Please note that this list is not exhaustive, it does not mean that the exercise in the final will cover exactly the same topics. I recall that the final exam will encompass all material covered in lectures, tutorials and labs. Clearly, this encompasses C language, and theoretical concepts such as time complexity.

Questions

Manual execution

1. Consider the following snippet:

```
int j = 30;
int k = 10;
if (j > k)
    j += k;
if (j > 30)
    j -= 20;
```

The final value of `j` is:

- a. 10
- b. 20
- c. 30
- d. 40
- e. 50
- f. 60

2. In the previous snippet, the final value of `k` is:

- a. 10
- b. 20
- c. 30
- d. 40
- e. 50
- f. 60

3. Consider the following snippet:

```
int c = 0;
int d = 4;
c = (d > 2) ? 2 : 3;
while(--d > 0){
    c++;
}
```

The final value of `c` is:

- a. 2
- b. 3
- c. 4
- d. 5
- e. 6
- f. 7

4. Consider the following snippet:

```
int b = 7;
int c = 0;
if (b > 9 && c++){
    c = 2;
} else if (b > 5 && ++c){
    b = 3;
} else {
    c = 4;
}
```

The final value of `b` is:

- a. 3
- b. 4
- c. 5
- d. 7
- e. 9
- f. 0

Programs output

5. What is the output of the execution of this program?

```
main(){
    int i;
    for(i=0; i<3; i++)
        printf("%d ", i+1);
}
```

- a. ``1 2 3``
- b. ``0 1 2``
- c. ``3 2 1``
- d. ``2 1 0``

6. Consider the program:

```

main(){
    int a,b;
    for(a=0;a<100;a++)
        for(b=0;b<100;b++)
            if (a + b == 10){
                printf("%d %d", a, b);
                return 0;
            }
    printf("Nothing");
    return 1;
}

```

Its output is:

- a. `0 10`
- b. `10 0`
- c. `1 9`
- d. `9 1`
- e. `Nothing`
- f. `0 0`

Filling a C program

10. We are asked to write a C program that prompt the user to enter a natural number n , then asks n natural numbers, and determines the maximum of all of them.

Here is an incomplete version of the program. Select the correct statement for each hole.

```

main(){
    int i,n,c,max;
    scanf("%d", &n);`
    for(max = 0,i=0 ; i<n ; i++){
        scanf("%d", &c);`
        if (c > max)
            // COMPLETE HERE
    }
    printf("%d\n", max);
}

```

Complete the missing hole in the code.

11.

The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding ones, usually starting with 0 and 1. For instance, 0, 1, 1, 2, 3, 5, 8, 13, 21, We are asked to write a C program that find the first pair of consecutive Fibonacci numbers in the sequence that are separated by a gap greater than a value given by the user. For instance, if the user enters 7, the program should show 13 and 21 which are at distance 8 which is greater than 7.

```
#include <stdio.h>

int main(){
    int gap;
    printf("Enter the gap: ")
    scanf("%d",&gap);
    int curr = 0;
    int next = 1;
    while (...){ // Complete the missing condition
        int old = curr;
        curr = next;
        next = curr + old;
    }
    printf("%d && %d", curr, next);
    return 0;
}
```

Writing a program

12. Write a C program that prompts the user to enter a natural number and the shows the sum of the square of the numbers between 0 and the number inserted by the user. For instance, if the user enters 5, the program shows 55 which is equal to $1*1 + 2*2 + 3*3 + 4*4 + 5*5$.
13. Write a program that reads characters by one by one from stdin and converts uppercase letters to lowercase and vice versa. Your program should handle input until EOF (End of File) is encountered.
14. Revise your previous solution to read entire strings of text from the user instead of processing characters one by one.
15. Consider the following code

```

#include <stdio.h>

void f(int a[], int b[], int size) {
    int i;
    for (i = 0; i < size; i++) {
        int j = a[i];
        printf("%d ", b[j]);
    }
    printf("\n");
}

int main() {
    int x[] = {0, 2, 4};
    int y[] = {1,2,3,4,5};
    f(x,y,3);
    f(y,x,3);
    return 0;
}

```

Describe the output produced by the two invocations of `f` in `main`.

16. Write a function that transposes a matrix. The function should allow to transpose any matrix of maximum number of columns `MAX_COLS` and maximum number of row `MAX_ROWS`.
17. Compute the worst-case time complexity of the function. Compute the type function $T(r,c)$ considering that ' r ' is the number of rows and ' c ' is the number of columns. Finally, express the complexity using big-O notation.
18. Write a C function that takes two arrays of different sizes as arguments: the first array is unsorted, and the second array is sorted. Design a function that checks whether each element in the first array exists in the second array and prints the result to `stdout`. Ensure that your solution operates with a time complexity of $O(k \log n)$ where k is the size of the first array and n is the size of the second array. If you use functions seen in lectures, there is no need of defining them
19. Give the output generated by the following code.

```

#include <stdio.h>

// Function to modify the array elements
void modify(int arr[], int size) {
    for (; size > 1; size--) {
        arr[size] *= 2;
    }
}

int main() {
    int myArray[] = {1, 2, 3, 4, 5};
    int size = 5;

    modify(myArray, size);

    for (int i = 0; i < size; i++) {
        printf("%d ", myArray[i]);
    }
    printf("\nsize: %d\n", size);

    return 0;
}

```

20. Consider the following program:

```

#include <stdio.h>

// Function to modify the array elements
void do_something(char str[]) {
    int i = 1;
    while (str[i] != '\0' && str[i] != '.'){
        i++;
    }
    str[i] = 0;
}

int main() {
    char str1[] = "Hello World!";
    char str2[] = "Another way. Hello World!";
    do_something(str1);
    printf("%s\n", str1);
    do_something(str2);
    printf("%s\n", str2);
    return 0;
}

```

- a. Indicates which is the output generated by the program.
- b. Consider that the function `do_something` is now used in this way

```
char str3[1];  
str3[0] = 'H';  
do_something(str3);  
printf("%s\n", str3);
```

Explain why this usage is incorrect.

21. Write a program that takes an integer argument from the command line, which specifies the size of an array. This integer should be a positive number and smaller than a specified maximum bound for the array size. Additionally, the program accepts a character (`u` or `l`) to determine how the array should be initialized: random uppercase letters for `u` or random lowercase letters for `l` . After initializing the array, the program sorts it using any of the sorting algorithms covered in lectures and then prints the sorted array. You don't need to implement the sorting algorithm.